

# **LLM-note**

Lecture notes on the theory of neural networks

Lianghong Mo

# Contents

|          |                                                                        |          |
|----------|------------------------------------------------------------------------|----------|
| <b>1</b> | <b>Three Questions in Deep-Learning Theory</b>                         | <b>3</b> |
| 1.1      | Introduction . . . . .                                                 | 3        |
| 1.1.1    | Definition of Neural Networks . . . . .                                | 4        |
| 1.1.2    | How Neural Networks Are Used . . . . .                                 | 5        |
| 1.1.3    | Three Main Topics . . . . .                                            | 6        |
| 1.2      | Topic I: Scaling Laws and Large-Network Limits . . . . .               | 6        |
| 1.2.1    | Basic Questions . . . . .                                              | 6        |
| 1.2.2    | Empirical Scaling Laws . . . . .                                       | 7        |
| 1.2.3    | Optimization, Approximation, and Data Errors . . . . .                 | 7        |
| 1.2.4    | Theoretical Approaches . . . . .                                       | 8        |
| 1.2.5    | Important Distinction: Scaling Is Not Just “More Parameters” . . . . . | 9        |
| 1.3      | Topic II: Feature Learning and Representation Learning . . . . .       | 10       |
| 1.3.1    | Linear Models . . . . .                                                | 10       |
| 1.3.2    | Neural Networks Learn the Features . . . . .                           | 10       |
| 1.3.3    | The Central Feature-Learning Question . . . . .                        | 11       |
| 1.3.4    | A Minimal Two-Layer Model . . . . .                                    | 11       |
| 1.3.5    | What Is a Semantic Feature? . . . . .                                  | 12       |
| 1.3.6    | Distributed Representations and Superposition . . . . .                | 12       |
| 1.3.7    | Feature Geometry . . . . .                                             | 13       |
| 1.3.8    | Transformers and In-Context Learning . . . . .                         | 13       |
| 1.4      | Topic III: Diffusion Models . . . . .                                  | 14       |
| 1.4.1    | Generative Modeling . . . . .                                          | 14       |
| 1.4.2    | Forward Diffusion Process . . . . .                                    | 14       |
| 1.4.3    | The Score Function . . . . .                                           | 15       |
| 1.4.4    | Why Is the Score Enough? . . . . .                                     | 15       |
| 1.4.5    | Learning the Score . . . . .                                           | 15       |
| 1.4.6    | Equivalent Noise-Prediction Formulation . . . . .                      | 16       |
| 1.4.7    | Probability-Flow ODE . . . . .                                         | 16       |
| 1.4.8    | Geometrical Picture . . . . .                                          | 17       |
| 1.4.9    | Main Questions for Diffusion Models . . . . .                          | 17       |
| 1.5      | Relations Between the Three Topics . . . . .                           | 17       |
| 1.5.1    | Scaling and Feature Learning . . . . .                                 | 17       |
| 1.5.2    | Feature Learning and Diffusion . . . . .                               | 18       |
| 1.5.3    | Scaling and Generative Modeling . . . . .                              | 18       |
| 1.6      | Big Picture . . . . .                                                  | 18       |

|          |                                                               |           |
|----------|---------------------------------------------------------------|-----------|
| <b>2</b> | <b>Scaling Limits of Neural Networks</b>                      | <b>19</b> |
| 2.1      | Basic setup . . . . .                                         | 19        |
| 2.2      | The scaling question . . . . .                                | 20        |
| 2.3      | Depth versus width: deep linear networks . . . . .            | 21        |
| 2.3.1    | Non-commuting limits . . . . .                                | 22        |
| 2.4      | Large-depth limits of ResNets . . . . .                       | 23        |
| 2.4.1    | The $1/\sqrt{L}$ limit: Neural SDE . . . . .                  | 24        |
| 2.4.2    | The $1/L$ limit: Neural ODE . . . . .                         | 24        |
| 2.4.3    | What lies between $1/\sqrt{L}$ and $1/L$ ? . . . . .          | 25        |
| 2.5      | Number of parameters versus number of data points . . . . .   | 25        |
| 2.5.1    | High-dimensional limit and the Marchenko–Pastur law . . . . . | 26        |
| 2.5.2    | Prediction risk and the optimal ridge parameter . . . . .     | 27        |
| 2.6      | Summary: the recurring idea . . . . .                         | 28        |

# Chapter 1

## Three Questions in Deep-Learning Theory

### Key ideas

- A neural network is a parametrized family of functions; what it learns is fixed jointly by the architecture, the data, and the optimization algorithm.
- **Scaling:** which simple effective theory emerges as  $N, n, L, W, C \rightarrow \infty$  (NTK, mean field, DMFT, Gaussian processes), and why these limits need not commute.
- **Feature learning:** a network learns the feature map  $\Phi_\theta(x)$  itself, not only its coefficients; features are directions in activation space and can be stored in superposition.
- **Diffusion:** learning the score  $\nabla_x \log \rho_t$  by denoising is enough to reverse a noising process and sample from  $\rho_{\text{data}}$ .

### 1.1 Introduction

A useful viewpoint is that a neural network is a *parametrized family of functions*. We choose an architecture, which determines a family

$$\{f_\theta : \mathcal{X} \rightarrow \mathcal{Y}\}_{\theta \in \Theta}, \quad (1.1)$$

and learning consists of selecting the parameter  $\theta$  using data.

Thus there are three conceptually distinct ingredients:

1. the **architecture**, which specifies the function class;
2. the **data**, which provide information about the unknown target;
3. the **optimization algorithm**, which determines which member of the function class is actually learned.

One of the central questions in the theory of neural networks is therefore

$$\boxed{\text{Architecture} + \text{Data} + \text{Optimization} \implies \text{Learned function} / \text{representation.}} \quad (1.2)$$

### 1.1.1 Definition of Neural Networks

#### 1.1.1.1 Fully connected neural networks

Consider an input

$$x \in \mathbb{R}^{N_0}. \quad (1.3)$$

Define

$$z^{(0)}(x) = x. \quad (1.4)$$

A fully connected neural network with  $L + 1$  layers is recursively defined by

$$z^{(\ell+1)}(x) = W^{(\ell+1)}\phi\left(z^{(\ell)}(x)\right) + b^{(\ell+1)}, \quad \ell = 0, \dots, L, \quad (1.5)$$

where

$$W^{(\ell+1)} \in \mathbb{R}^{N_{\ell+1} \times N_\ell}, \quad b^{(\ell+1)} \in \mathbb{R}^{N_{\ell+1}}, \quad (1.6)$$

and  $\phi$  is a nonlinear activation function applied componentwise.<sup>1</sup>

Typical choices include

$$\text{ReLU}(x) = \max(0, x), \quad (1.7)$$

$$\tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}, \quad (1.8)$$

$$\text{GELU}(x) = x\Phi(x), \quad (1.9)$$

where  $\Phi$  is the standard Gaussian cumulative distribution function.

The output of the network is

$$f_\theta(x) \equiv z^{(L+1)}(x), \quad (1.10)$$

where the parameter set is

$$\theta = \left\{ W^{(1)}, \dots, W^{(L+1)}, b^{(1)}, \dots, b^{(L+1)} \right\}. \quad (1.11)$$

The nonlinear activation is essential. Without  $\phi$ , a composition of linear layers would still be only a linear transformation:

$$W^{(L)}W^{(L-1)} \dots W^{(1)}x = W_{\text{eff}}x. \quad (1.12)$$

#### 1.1.1.2 Residual neural networks

Instead of replacing the current representation at every layer, a residual network learns a correction:

$$z^{(\ell+1)} = z^{(\ell)} + F_{\theta_\ell}\left(z^{(\ell)}\right). \quad (1.13)$$

A typical residual block is

$$F_{\theta_\ell}(z) = W_2^{(\ell)}\phi\left(W_1^{(\ell)}z + b_1^{(\ell)}\right) + b_2^{(\ell)}. \quad (1.14)$$

Therefore,

$$z^{(\ell+1)} = z^{(\ell)} + W_2^{(\ell)}\phi\left(W_1^{(\ell)}z^{(\ell)} + b_1^{(\ell)}\right) + b_2^{(\ell)}. \quad (1.15)$$

---

<sup>1</sup>With this convention  $\phi$  also acts on the raw input  $z^{(0)} = x$ . Lecture 2 uses the equally common convention  $z^{(1)} = W^{(1)}x + b^{(1)}$ ; nothing below depends on the choice.

The important physical picture is that each layer makes only an incremental update,

$$\Delta z^{(\ell)} = F_{\theta_\ell}(z^{(\ell)}). \quad (1.16)$$

In a formal continuum-depth limit, a residual network resembles a dynamical system,

$$\frac{dz(t)}{dt} = F_{\theta(t)}(z(t)), \quad (1.17)$$

which motivates the connection between residual networks and differential equations.

### 1.1.2 How Neural Networks Are Used

Suppose there is an unknown target function

$$f : \mathcal{X} \rightarrow \mathcal{Y}. \quad (1.18)$$

We observe a training dataset

$$\mathcal{D} = \{(x_i, y_i)\}_{i=1}^n, \quad y_i = f(x_i), \quad (1.19)$$

where  $n$  denotes the number of training samples.

The standard supervised-learning procedure is:

1. **Data:** observe

$$\{(x_i, f(x_i))\}_{i=1}^n. \quad (1.20)$$

The goal is to infer the unknown function  $f$  from finitely many observations.

2. **Initialization:** choose an initial parameter

$$\theta_0 \sim P_{\text{init}}. \quad (1.21)$$

3. **Optimization:** minimize a loss function. For regression, a common choice is

$$\mathcal{L}(\theta) = \frac{1}{2n} \sum_{i=1}^n \|f(x_i) - f_\theta(x_i)\|^2. \quad (1.22)$$

Gradient descent gives

$$\theta_{t+1} = \theta_t - \eta \nabla_\theta \mathcal{L}(\theta_t), \quad (1.23)$$

where  $\eta$  is the learning rate.

In practice one usually uses stochastic gradient descent,

$$\theta_{t+1} = \theta_t - \eta \widehat{\nabla_\theta \mathcal{L}}, \quad (1.24)$$

where the gradient is estimated from a minibatch.

4. **Generalization / test:** for a new point  $x_{\text{new}}$  that was not in the training set, ask whether

$$f(x_{\text{new}}) \approx f_{\theta_{\text{learned}}}(x_{\text{new}}). \quad (1.25)$$

A neural network is therefore not useful merely because it can fit the training data. The central problem is whether it can *generalize*.

### 1.1.3 Three Main Topics

We focus on three broad theoretical questions:

1. **Scaling laws and large-network limits:** What happens as the model size, dataset size, and computational budget become very large?
2. **Feature learning and representation learning:** What internal representations does the neural network learn, and why do these representations help generalization?
3. **Generative models, especially diffusion models:** How can a neural network learn an entire high-dimensional probability distribution and generate new samples from it?

## 1.2 Topic I: Scaling Laws and Large-Network Limits

### 1.2.1 Basic Questions

Modern neural networks involve several potentially large parameters:

$$N = \text{number of model parameters}, \quad (1.26)$$

$$n = \text{number of training samples}, \quad (1.27)$$

$$L = \text{depth}, \quad (1.28)$$

$$W = \text{width}, \quad (1.29)$$

$$C = \text{computational budget}. \quad (1.30)$$

The theoretical problem resembles statistical mechanics: instead of studying one finite neural network in detail, we would like to identify simple laws that emerge in appropriate large-scale limits.

Some central questions are:

1. What happens when

$$N, n, L, W, C \rightarrow \infty? \quad (1.31)$$

These limits need not commute. For example,

$$\lim_{n \rightarrow \infty} \lim_{W \rightarrow \infty} \quad (1.32)$$

may describe a different regime from

$$\lim_{W \rightarrow \infty} \lim_{n \rightarrow \infty}. \quad (1.33)$$

Therefore the interesting object is often a *joint scaling limit*, analogous to a thermodynamic limit.

2. Does a complicated neural network reduce to a simpler effective theory in such a limit?
3. Can performance at very large scale be predicted from experiments at relatively small scale?

4. Which resource is the bottleneck:

$$\text{model size,} \quad \text{data,} \quad \text{optimization,} \quad \text{compute?} \quad (1.34)$$

5. When does increasing the model size actually improve performance, and when does performance saturate?

### 1.2.2 Empirical Scaling Laws

Empirically, many machine-learning systems approximately obey power-law scaling over a substantial range of scales.

For example, test loss may behave as

$$\boxed{\mathcal{L}(C) = \mathcal{L}_\infty + AC^{-\beta}}, \quad (1.35)$$

where

$$C = \text{compute}, \quad (1.36)$$

$\beta > 0$  is a scaling exponent, and  $\mathcal{L}_\infty$  is an irreducible loss floor.

Similarly one may study

$$\mathcal{L}(N) \approx \mathcal{L}_\infty + A_N N^{-\alpha_N}, \quad (1.37)$$

$$\mathcal{L}(n) \approx \mathcal{L}_\infty + A_n n^{-\alpha_n}. \quad (1.38)$$

More generally,

$$\mathcal{L} = \mathcal{L}(N, n, C, \dots). \quad (1.39)$$

The interesting theoretical questions are not only whether these power laws exist, but also:

1. why power laws emerge;
2. whether the exponents are universal;
3. how the exponents depend on the data distribution and architecture;
4. how to optimally allocate a fixed computational budget between increasing  $N$ , increasing  $n$ , and training for longer.

### 1.2.3 Optimization, Approximation, and Data Errors

It is useful to conceptually decompose the total error into several pieces:

$$\mathcal{E}_{\text{total}} \sim \mathcal{E}_{\text{approx}} + \mathcal{E}_{\text{estimation}} + \mathcal{E}_{\text{optimization}}. \quad (1.40)$$

Here:

- $\mathcal{E}_{\text{approx}}$  measures whether the architecture is expressive enough to represent the desired function;
- $\mathcal{E}_{\text{estimation}}$  arises because the training dataset is finite;
- $\mathcal{E}_{\text{optimization}}$  measures whether the optimizer succeeds in finding a good parameter configuration.

Scaling the number of parameters only directly attacks the first of these. Therefore increasing model size alone need not always improve performance.

## 1.2.4 Theoretical Approaches

### 1.2.4.1 Neural Tangent Kernel

Consider the network output

$$f_\theta(x). \quad (1.41)$$

Linearize it around the initialization  $\theta_0$ :

$$f_\theta(x) \approx f_{\theta_0}(x) + \nabla_\theta f_{\theta_0}(x)^T (\theta - \theta_0). \quad (1.42)$$

Define the Neural Tangent Kernel

$$K_{\text{NTK}}(x, x') = \nabla_\theta f_{\theta_0}(x)^T \nabla_\theta f_{\theta_0}(x'). \quad (1.43)$$

In an appropriate infinite-width limit, the NTK becomes approximately deterministic and changes very little during training.

The nonlinear neural-network training problem is then reduced to an effective kernel regression problem.

The physical picture is

$$\boxed{\text{very wide network} \Rightarrow \text{approximately fixed features} + \text{linearized dynamics.}} \quad (1.44)$$

This regime is often called the *lazy-training regime*.

A major limitation is that realistic neural networks often exhibit substantial *feature learning*, whereas the simplest NTK limit freezes the features.

### 1.2.4.2 Mean-Field Theory

Another large-width limit treats individual neurons as interacting particles.

Consider a two-layer network

$$f_m(x) = \frac{1}{m} \sum_{j=1}^m a_j \sigma(w_j^T x). \quad (1.45)$$

Instead of following every neuron separately, define an empirical distribution of neuron parameters,

$$\rho_m(a, w) = \frac{1}{m} \sum_{j=1}^m \delta(a - a_j) \delta(w - w_j). \quad (1.46)$$

As

$$m \rightarrow \infty, \quad (1.47)$$

one can sometimes derive a deterministic evolution equation for

$$\rho(a, w, t). \quad (1.48)$$

Schematically,

$$\partial_t \rho = \mathcal{F}[\rho]. \quad (1.49)$$

This is analogous to the passage

$$\text{many particles} \rightarrow \text{continuum density} \quad (1.50)$$

in statistical mechanics.

Unlike the simplest NTK regime, the distribution of features  $w$  can evolve, so mean-field theory can describe genuine feature learning.

### 1.2.4.3 Dynamical Mean-Field Theory

For sufficiently complicated random neural networks, especially deep networks or recurrent systems, one can seek an effective stochastic description of a typical neuron or feature.

The philosophy is analogous to dynamical mean-field theory in many-body physics:

$$\boxed{\text{high-dimensional interacting system} \rightarrow \text{self-consistent effective single-site dynamics.}} \quad (1.51)$$

Instead of tracking every neuron explicitly, one derives self-consistency equations for quantities such as

$$C(t, t') = \langle z(t)z(t') \rangle, \quad (1.52)$$

$$R(t, t') = \frac{\delta \langle z(t) \rangle}{\delta h(t')}. \quad (1.53)$$

These quantities play roles analogous to correlation and response functions in statistical physics.

### 1.2.4.4 Bayesian Inference

A different viewpoint is to place a prior distribution on the network parameters,

$$P(\theta), \quad (1.54)$$

and use Bayes' rule:

$$P(\theta | \mathcal{D}) = \frac{P(\mathcal{D} | \theta)P(\theta)}{P(\mathcal{D})}. \quad (1.55)$$

Predictions are then obtained by averaging over the posterior:

$$P(y_* | x_*, \mathcal{D}) = \int d\theta P(y_* | x_*, \theta)P(\theta | \mathcal{D}). \quad (1.56)$$

For some infinitely wide neural networks, the prior over functions becomes a Gaussian process,

$$f(x) \sim \text{GP}(0, K(x, x')). \quad (1.57)$$

This provides another analytically tractable infinite-width limit.

## 1.2.5 Important Distinction: Scaling Is Not Just “More Parameters”

When discussing scaling, one should distinguish several directions:

$$\text{model scaling : } N \rightarrow \infty, \quad (1.58)$$

$$\text{data scaling : } n \rightarrow \infty, \quad (1.59)$$

$$\text{compute scaling : } C \rightarrow \infty, \quad (1.60)$$

$$\text{context / horizon scaling : } T \rightarrow \infty. \quad (1.61)$$

Good scaling often requires these quantities to grow together.

Thus,

$$\boxed{\text{Scaling up the model} \neq \text{scaling up the learning problem.}} \quad (1.62)$$

## 1.3 Topic II: Feature Learning and Representation Learning

### 1.3.1 Linear Models

Before studying neural networks, consider a linear model in a fixed feature space:

$$f_{\theta}(x) = \theta^T \Phi(x) = \sum_{j=1}^M \theta_j \varphi_j(x), \quad (1.63)$$

where

$$\Phi(x) = \begin{pmatrix} \varphi_1(x) \\ \vdots \\ \varphi_M(x) \end{pmatrix} \quad (1.64)$$

is a feature map.

The important point is that the features

$$\varphi_j(x) \quad (1.65)$$

are fixed. Learning only determines their coefficients  $\theta_j$ .

Therefore a linear model solves

fixed representation + learned linear coefficients.

(1.66)

### 1.3.2 Neural Networks Learn the Features

Consider the final layer of a neural network:

$$f_i(x; \theta) = \sum_{j=1}^{N_L} W_{ij}^{(L+1)} \phi_j(x; \theta), \quad (1.67)$$

where

$$\phi_j(x; \theta) = \phi\left(z_j^{(L)}(x; \theta)\right). \quad (1.68)$$

From the perspective of the final layer, this looks exactly like a linear model:

$$f(x) = W^{(L+1)} \Phi_{\theta}(x). \quad (1.69)$$

The crucial difference is

$\Phi_{\theta}(x)$  itself depends on the learned parameters.

(1.70)

Thus neural-network training does two things simultaneously:

1. it learns how to combine features;
2. it learns what the features themselves should be.

This is the basic meaning of *representation learning*.

### 1.3.3 The Central Feature-Learning Question

The basic theoretical problem is:

$$\boxed{\text{Data} + \text{Architecture} + \text{Optimization} + \text{Initialization} \rightarrow \text{Learned features.}} \quad (1.71)$$

More explicitly, we would like to understand

$$\Phi_{\theta_t}(x) \quad (1.72)$$

as a function of training time  $t$ .

Questions include:

1. Which features emerge first during training?
2. Which properties of the data determine the learned representation?
3. Why are some features reused across many tasks?
4. When are representations sparse or distributed?
5. When does the network discover semantic concepts?
6. How do learned representations change with width and depth?

### 1.3.4 A Minimal Two-Layer Model

Consider

$$f(x; t) = \frac{1}{\sqrt{m}} \sum_{j=1}^m a_j(t) \sigma(w_j(t)^T x). \quad (1.73)$$

There are two qualitatively different kinds of learning.

If only

$$a_j(t) \quad (1.74)$$

change while

$$w_j(t) \approx w_j(0), \quad (1.75)$$

then the network is approximately learning a linear combination of fixed features

$$\sigma(w_j(0)^T x). \quad (1.76)$$

By contrast, if

$$w_j(t) \quad (1.77)$$

move substantially, the features themselves change:

$$\sigma(w_j(0)^T x) \longrightarrow \sigma(w_j(t)^T x). \quad (1.78)$$

This is genuine feature learning.

This also makes the connection with Topic I clear:

$$\boxed{\text{NTK} / \text{lazy limit} \approx \text{weak feature learning,}} \quad (1.79)$$

whereas other scaling limits can preserve nontrivial feature dynamics.

**Remark (parametrization).** The prefactor  $1/\sqrt{m}$  used here is the NTK scaling, while the mean-field model of Section 1.2.4.2 uses  $1/m$ . With  $1/\sqrt{m}$  and an  $O(1)$  learning rate, each  $w_j$  moves by only  $O(1/\sqrt{m})$  during training, so the features freeze as  $m \rightarrow \infty$ . With  $1/m$  and a learning rate scaled up by  $m$ , each  $w_j$  moves by  $O(1)$ . The choice of normalization is therefore what selects lazy versus feature-learning behavior.

### 1.3.5 What Is a Semantic Feature?

Suppose a vision model is given an image of the Golden Gate Bridge.

At the raw-input level, the image consists only of pixel values:

$$x = (x_1, x_2, \dots, x_N). \quad (1.80)$$

A low-level feature might detect:

$$\text{edges, colors, textures.} \quad (1.81)$$

A higher-level representation may encode concepts such as

$$\text{bridge, suspension bridge, Golden Gate Bridge, San Francisco.} \quad (1.82)$$

These are called *semantic features* because they correspond to meaningful concepts rather than simple local properties of the input.

An important question is whether such concepts correspond to individual neurons or to directions in a high-dimensional representation space.

### 1.3.6 Distributed Representations and Superposition

A naive picture is

$$\text{one neuron} \leftrightarrow \text{one concept.} \quad (1.83)$$

However, modern neural networks often appear to use distributed representations.

A feature may correspond to a direction

$$v_\alpha \in \mathbb{R}^d \quad (1.84)$$

in activation space, rather than to a single neuron.

An activation vector may schematically be written as

$$h(x) \approx \sum_{\alpha} c_{\alpha}(x) v_{\alpha}. \quad (1.85)$$

If the number of useful features is larger than the activation dimension, the network may store multiple features in non-orthogonal directions:

$$v_{\alpha}^T v_{\beta} \neq 0. \quad (1.86)$$

This phenomenon is often called *superposition*.

The rough idea is

$$\boxed{\text{number of useful features} > \text{number of available neurons}} \quad (1.87)$$

so the network compresses many features into a smaller-dimensional space.

This can lead to *polysemantic* neurons, where one neuron participates in representing several apparently unrelated concepts.

### 1.3.7 Feature Geometry

Rather than analyzing individual neurons, it is often more useful to study the geometry of representation space.

Let

$$h_\theta(x) \in \mathbb{R}^d \quad (1.88)$$

denote the learned representation.

Then one may study distances

$$d_\theta(x, x') = \|h_\theta(x) - h_\theta(x')\|, \quad (1.89)$$

similarities

$$K_\theta(x, x') = h_\theta(x)^T h_\theta(x'), \quad (1.90)$$

or covariance matrices

$$C = \mathbb{E}_x [h_\theta(x) h_\theta(x)^T]. \quad (1.91)$$

The spectrum of  $C$  provides information about the effective dimensionality of the learned representation.

### 1.3.8 Transformers and In-Context Learning

Feature learning becomes particularly interesting in transformers.

A transformer maps a sequence

$$x_1, \dots, x_T \quad (1.92)$$

to internal representations

$$h_1^{(\ell)}, \dots, h_T^{(\ell)}. \quad (1.93)$$

Self-attention performs

$$q_i = W_Q h_i, \quad (1.94)$$

$$k_j = W_K h_j, \quad (1.95)$$

$$v_j = W_V h_j, \quad (1.96)$$

followed by

$$\text{Attention}(i) = \sum_j A_{ij} v_j, \quad (1.97)$$

with

$$A_{ij} = \frac{\exp(q_i^T k_j / \sqrt{d})}{\sum_{j'} \exp(q_i^T k_{j'} / \sqrt{d})}. \quad (1.98)$$

Unlike a fixed feature map, the representation of one token depends on the other tokens in the context.

This raises the question of *in-context learning*: how can the forward pass of a trained transformer implement something that resembles a learning algorithm?

Schematically,

$$\text{context examples} \rightarrow \text{internal representation} \rightarrow \text{new prediction}. \quad (1.99)$$

One viewpoint is that training does not merely teach the model a fixed input-output function; it can teach the model an *algorithm for extracting a task from context*.

## 1.4 Topic III: Diffusion Models

### 1.4.1 Generative Modeling

Suppose the data are drawn from an unknown high-dimensional distribution

$$x_0 \sim \rho_0(x). \quad (1.100)$$

For example,  $x_0$  may be an image.

The goal of a generative model is not merely to predict a label. Instead, we want to generate a new sample

$$x_{\text{new}} \sim \rho_0. \quad (1.101)$$

The main difficulty is that  $\rho_0(x)$  can be an extremely complicated distribution in a very high-dimensional space.

### 1.4.2 Forward Diffusion Process

Diffusion models introduce a continuous family of distributions

$$\rho_t, \quad t \in [0, T], \quad (1.102)$$

such that

$$\rho_{t=0} = \rho_{\text{data}}, \quad (1.103)$$

while

$$\rho_T \approx \mathcal{N}(0, I). \quad (1.104)$$

The forward process gradually adds Gaussian noise.

A common parametrization is

$$\boxed{x_t = \alpha_t x_0 + \sigma_t \epsilon, \quad \epsilon \sim \mathcal{N}(0, I).} \quad (1.105)$$

Initially,

$$\alpha_0 \approx 1, \quad \sigma_0 \approx 0, \quad (1.106)$$

so

$$x_t \approx x_0. \quad (1.107)$$

At sufficiently large  $t$ ,

$$\alpha_T \approx 0, \quad \sigma_T \approx 1, \quad (1.108)$$

so

$$x_T \approx \epsilon \sim \mathcal{N}(0, I). \quad (1.109)$$

Thus,

$$\boxed{\text{complicated data distribution} \rightarrow \text{simple Gaussian distribution.}} \quad (1.110)$$

### 1.4.3 The Score Function

Define the score of the intermediate distribution  $\rho_t$  by

$$\boxed{s_t(x) = \nabla_x \log \rho_t(x)}. \quad (1.111)$$

The score is a vector field in data space.

Intuitively,  $s_t(x)$  points toward directions in which the probability density increases most rapidly.

For a one-dimensional Gaussian,

$$\rho(x) \propto e^{-x^2/(2\sigma^2)}, \quad (1.112)$$

we have

$$s(x) = \frac{d}{dx} \log \rho(x) = -\frac{x}{\sigma^2}. \quad (1.113)$$

Therefore the score points back toward the high-probability region near  $x = 0$ .

### 1.4.4 Why Is the Score Enough?

Suppose the forward diffusion is described by the stochastic differential equation

$$dx = f(x, t) dt + g(t) dW_t. \quad (1.114)$$

The evolution of the probability density is described by a Fokker–Planck equation.

Remarkably, the corresponding reverse-time stochastic dynamics can be written in terms of the score:

$$dx = \left[ f(x, t) - g(t)^2 \nabla_x \log \rho_t(x) \right] dt + g(t) d\bar{W}_t, \quad (1.115)$$

where the equation is integrated from  $t = T$  back toward  $t = 0$ .

Therefore, if we know

$$s_t(x) = \nabla_x \log \rho_t(x) \quad (1.116)$$

for every  $t$ , we can reverse the noising process.

The central idea is consequently

$$\boxed{\text{learn the score} \implies \text{reverse diffusion} \implies \text{generate data.}} \quad (1.117)$$

### 1.4.5 Learning the Score

Introduce a neural network

$$s_\theta(x, t) \quad (1.118)$$

that approximates

$$\nabla_x \log \rho_t(x). \quad (1.119)$$

For the Gaussian corruption

$$x_t = \alpha_t x_0 + \sigma_t \epsilon, \quad (1.120)$$

the conditional distribution is

$$p(x_t \mid x_0) = \mathcal{N}(\alpha_t x_0, \sigma_t^2 I). \quad (1.121)$$

Its score is known exactly:

$$\nabla_{x_t} \log p(x_t | x_0) = -\frac{x_t - \alpha_t x_0}{\sigma_t^2}. \quad (1.122)$$

Since

$$x_t - \alpha_t x_0 = \sigma_t \epsilon, \quad (1.123)$$

this becomes

$$\nabla_{x_t} \log p(x_t | x_0) = -\frac{\epsilon}{\sigma_t}. \quad (1.124)$$

Therefore one can train the network using denoising score matching:

$$\mathcal{L}_{\text{score}}(\theta) = \mathbb{E}_{t, x_0, \epsilon} \left[ \lambda(t) \left\| s_\theta(x_t, t) + \frac{\epsilon}{\sigma_t} \right\|^2 \right]. \quad (1.125)$$

Thus the training problem can be reduced to a supervised regression problem with artificially generated Gaussian noise.

#### 1.4.6 Equivalent Noise-Prediction Formulation

Instead of directly predicting the score, many diffusion models predict the noise

$$\epsilon_\theta(x_t, t). \quad (1.126)$$

Since

$$s_t(x_t) \sim -\frac{\epsilon}{\sigma_t}, \quad (1.127)$$

the two representations are directly related:

$$s_\theta(x_t, t) = -\frac{\epsilon_\theta(x_t, t)}{\sigma_t}. \quad (1.128)$$

The corresponding loss is approximately

$$\mathcal{L}_\epsilon = \mathbb{E} \left[ \|\epsilon - \epsilon_\theta(x_t, t)\|^2 \right]. \quad (1.129)$$

#### 1.4.7 Probability-Flow ODE

The stochastic reverse dynamics are not the only way to generate samples.

The same family of marginal distributions  $\rho_t$  can also be reproduced by a deterministic probability-flow ODE,

$$\frac{dx}{dt} = f(x, t) - \frac{1}{2}g(t)^2 s_t(x). \quad (1.130)$$

Thus diffusion models can be understood either as

$$\text{stochastic reverse dynamics} \quad (1.131)$$

or as

$$\text{deterministic transport through probability space.} \quad (1.132)$$

This also provides a connection to flow-based generative models and flow matching.

### 1.4.8 Geometrical Picture

At large noise,

$$\rho_T(x) \approx \mathcal{N}(0, I), \quad (1.133)$$

which is simple.

As  $t$  decreases, the score field gradually guides samples toward the high-probability regions of the data distribution.

Schematically,

$$\text{Gaussian noise} \rightarrow \text{coarse structure} \rightarrow \text{objects} \rightarrow \text{fine details}. \quad (1.134)$$

In this sense, the neural network learns a time-dependent vector field

$$s_\theta(x, t) \quad (1.135)$$

that describes the geometry of the probability distribution at different noise scales.

### 1.4.9 Main Questions for Diffusion Models

Important theoretical questions include:

1. Why is the score easier to learn than the probability density itself?
2. Why does adding noise make the learning problem easier?
3. How accurately must the score be learned in order to generate accurate samples?
4. How does error accumulate during the reverse process?
5. Why do diffusion models scale so effectively with model size and data?
6. What representations are learned by the denoising network?
7. What is the relation between diffusion models, optimal transport, normalizing flows, and flow matching?
8. How many reverse-time steps are fundamentally necessary?
9. Which aspects of the data distribution are learned at large noise and which are learned only at small noise?

## 1.5 Relations Between the Three Topics

Although scaling laws, feature learning, and diffusion models initially look like separate topics, they are closely connected.

### 1.5.1 Scaling and Feature Learning

The infinite-width limit used in NTK theory can suppress feature learning:

$$\text{width} \rightarrow \infty \quad \Rightarrow \quad \text{features approximately frozen}. \quad (1.136)$$

Therefore an important question is which scaling limit retains the rich feature learning observed in practical neural networks.

This illustrates why different limits need not commute.

### 1.5.2 Feature Learning and Diffusion

A diffusion model is trained to estimate

$$s_t(x) = \nabla_x \log \rho_t(x) \quad (1.137)$$

at many noise levels.

To do this, the network must learn useful internal representations of the data.

At large  $t$ , these representations may encode coarse global structure, whereas at small  $t$  they must encode increasingly fine details.

Thus diffusion models provide a natural laboratory for studying how representations depend on scale.

### 1.5.3 Scaling and Generative Modeling

A generative model must represent an entire probability distribution rather than merely a decision boundary.

This raises particularly sharp scaling questions:

$$\text{How much model capacity and data are required to represent } \rho_{\text{data}}? \quad (1.138)$$

One may therefore study scaling laws such as

$$\mathcal{L}_{\text{gen}} = \mathcal{L}(N, n, C), \quad (1.139)$$

and ask whether the same scaling ideas observed in supervised learning also govern modern generative models.

## 1.6 Big Picture

The three topics can be summarized as follows.

| Topic            | Central Question                                                       |
|------------------|------------------------------------------------------------------------|
| Scaling          | What simple theory emerges when neural networks become very large?     |
| Feature learning | What internal representation does training discover?                   |
| Diffusion        | How can a neural network learn and sample from an entire distribution? |

A common theme is the search for an effective theory of learning:

$$\boxed{\text{microscopic parameter dynamics} \longrightarrow \text{macroscopic laws of learning.}} \quad (1.140)$$

From a statistical-physics perspective, neural networks are particularly interesting because they are large interacting dynamical systems whose effective degrees of freedom, representations, and scaling laws are themselves learned from data.

## Chapter 2

# Scaling Limits of Neural Networks

### Key ideas

- Large networks are controlled by a few *dimensionless ratios*, not by the raw sizes. A scaling limit is specified by the path to infinity together with how the hyperparameters scale.
- **Deep linear network:**  $\log(\|z^{(L)}\|/\|x\|) \rightarrow \mathcal{N}(-\xi/2, \xi/2)$  with effective depth  $\xi = L/N$ ; the width-first and depth-first limits differ.
- **ResNet:**  $s_L \sim L^{-1/2}$  is critical for iid layers and gives a Neural SDE;  $s_L \sim L^{-1}$  requires weights that vary smoothly across depth and gives a Neural ODE; correlated weights with Hurst exponent  $H$  interpolate via  $s_L \sim L^{-H}$ .
- **Ridge regression:** at  $\gamma = d/n$  the Marchenko–Pastur law gives the exact asymptotic risk, minimized at  $\lambda_* = \gamma/\text{SNR}$ .

### 2.1 Basic setup

**Definition (MLP).** For an input

$$x \in \mathbb{R}^{N_0},$$

a multilayer perceptron is defined recursively by

$$z^{(1)}(x) = W^{(1)}x + b^{(1)} \in \mathbb{R}^{N_1}, \quad (2.1)$$

$$z^{(\ell+1)}(x) = W^{(\ell+1)}\phi\left(z^{(\ell)}(x)\right) + b^{(\ell+1)}, \quad \ell = 1, \dots, L-1. \quad (2.2)$$

Here  $N_\ell$  is the width of layer  $\ell$ , while  $L$  is the depth of the network. In the simplest constant-width setting,

$$N_1 = \dots = N_L = N.$$

**Definition (ResNet).** A residual network takes the form

$$z^{(\ell+1)} = z^{(\ell)} + s_L F^{(\ell)}\left(z^{(\ell)}\right), \quad (2.3)$$

where, for example,

$$F^{(\ell)}(z) = W_2^{(\ell)} \phi(W_1^{(\ell)} z + b_1^{(\ell)}) + b_2^{(\ell)}. \quad (2.4)$$

The factor  $s_L$  controls how the residual branch must scale as the depth  $L$  increases.

## 2.2 The scaling question

A central question is:

*What limiting dynamics arise when the model size, data size, and training time all become large?*

There are many possible large parameters.

**Model scales.**

- depth  $L$ ,
- width  $N$ ,
- number of attention heads  $H$ ,
- embedding dimension, number of parameters, etc.

**Data and optimization scales.**

- number of data points  $n$ ,
- batch size  $B$ ,
- number of optimization steps  $T$ .

It is not enough to simply say that all of these quantities go to infinity. One must specify the *path* to infinity and how the hyperparameters scale:

$$\mathcal{H} = \{\eta, \sigma, B, \lambda, \dots\} = \mathcal{H}(L, N, n, T, \dots). \quad (2.5)$$

**Motivations.**

1. Identify a small number of macroscopic variables that describe learning, analogous to thermodynamic variables in statistical physics. Examples include activation norms, feature covariances, kernel spectra, and training loss.
2. Identify parameterizations for which scaling is predictable: when the model becomes larger, one would like quantities such as the optimal learning rate or initialization scale to remain controlled. This is the idea behind *hyperparameter transfer*.

We first study three simple examples:

1. depth  $L$  versus width  $N$  in a fully connected network;
2. the large-depth limit of a ResNet;
3. parameter dimension  $d$  versus number of data points  $n$ .

## 2.3 Depth versus width: deep linear networks

We first remove all nonlinear complications and take

$$\phi(z) = z, \quad b^{(\ell)} = 0.$$

Then

$$z^{(L)}(x) = W^{(L)}W^{(L-1)} \dots W^{(1)}x. \quad (2.6)$$

Assume all hidden layers have width  $N$ , and initialize

$$W_{ij}^{(\ell)} \stackrel{\text{iid}}{\sim} \mathcal{N}\left(0, \frac{1}{N}\right). \quad (2.7)$$

The  $1/N$  variance scaling is the natural random-matrix scaling: for a fixed vector  $v$ ,

$$\mathbb{E} \left[ \|W^{(\ell)}v\|_2^2 \mid v \right] = \|v\|_2^2. \quad (2.8)$$

More precisely, rotational invariance gives

$$\frac{\|W^{(\ell)}v\|_2^2}{\|v\|_2^2} \stackrel{d}{=} \frac{1}{N} \chi_N^2. \quad (2.9)$$

Since different layers are independent,

$$\frac{\|z^{(L)}(x)\|_2}{\|x\|_2} \stackrel{d}{=} \prod_{\ell=1}^L \sqrt{\frac{\chi_{N,\ell}^2}{N}}, \quad (2.10)$$

where the  $\chi_{N,\ell}^2$ 's are independent.

Taking the logarithm turns the product into a sum:

$$\log \frac{\|z^{(L)}(x)\|_2}{\|x\|_2} = \frac{1}{2} \sum_{\ell=1}^L \log \left( \frac{\chi_{N,\ell}^2}{N} \right). \quad (2.11)$$

For large  $N$ ,

$$\mathbb{E} \left[ \log \left( \frac{\chi_N^2}{N} \right) \right] = -\frac{1}{N} + O(N^{-2}), \quad (2.12)$$

$$\text{Var} \left[ \log \left( \frac{\chi_N^2}{N} \right) \right] = \frac{2}{N} + O(N^{-2}). \quad (2.13)$$

Therefore,

$$\log \frac{\|z^{(L)}(x)\|_2}{\|x\|_2} \approx \mathcal{N}\left(-\frac{L}{2N}, \frac{L}{2N}\right). \quad (2.14)$$

This immediately identifies the natural dimensionless parameter

$$\boxed{\xi = \frac{L}{N}} \quad (2.15)$$

which acts as an *effective depth*.

If

$$L, N \rightarrow \infty, \quad \frac{L}{N} \rightarrow \xi,$$

then

$$\log \frac{\|z^{(L)}(x)\|_2}{\|x\|_2} \Rightarrow \mathcal{N}\left(-\frac{\xi}{2}, \frac{\xi}{2}\right). \quad (2.16)$$

Thus the norm becomes approximately log-normal.

**Important subtlety.** Although

$$\mathbb{E}\|z^{(L)}(x)\|_2^2 = \|x\|_2^2 \quad (2.17)$$

for every  $L$ , the *typical* norm decreases:

$$\mathbb{E} \left[ \log \frac{\|z^{(L)}(x)\|_2}{\|x\|_2} \right] < 0.$$

The conserved second moment is maintained by increasingly rare realizations with large norms. This is the distinction between an averaged quantity and a typical realization.

### 2.3.1 Non-commuting limits

The two limits  $N \rightarrow \infty$  and  $L \rightarrow \infty$  do not commute.

**1. Width first: free-probability (FP) regime.** Take

$$N \rightarrow \infty \quad \text{with fixed } L.$$

Then

$$\frac{\chi_N^2}{N} \rightarrow 1$$

and hence, for a fixed input,

$$\frac{\|z^{(L)}(x)\|}{\|x\|} \rightarrow 1. \quad (2.18)$$

The product matrix itself is *not* the identity; rather, its large- $N$  singular-value spectrum is naturally described using free probability.

**2. Depth first: multiplicative ergodic theorem (MET) regime.** For fixed  $N$  and

$$L \rightarrow \infty,$$

the logarithm of the norm is a sum of independent random increments. Therefore

$$\frac{1}{L} \log \frac{\|z^{(L)}(x)\|}{\|x\|} \rightarrow \frac{1}{2} \mathbb{E} \log \left( \frac{\chi_N^2}{N} \right) < 0. \quad (2.19)$$

This is the multiplicative-ergodic/Lyapunov-exponent picture.

**3. Double-scaling regime.** The interpolation between these limits is

$$N, L \rightarrow \infty, \quad \frac{L}{N} \rightarrow \xi. \quad (2.20)$$

Hence  $L/N$ , rather than  $L$  alone, is the natural effective depth.

## 2.4 Large-depth limits of ResNets

Consider first the simple linear residual model

$$z^{(\ell+1)} = \left( I + s_L W^{(\ell)} \right) z^{(\ell)}, \quad (2.21)$$

where

$$W_{ij}^{(\ell)} \sim \mathcal{N}\left(0, \frac{1}{N}\right)$$

independently across layers.

Conditioned on  $z^{(\ell)}$ ,

$$\mathbb{E} \left[ \|z^{(\ell+1)}\|^2 \mid z^{(\ell)} \right] = \|z^{(\ell)}\|^2 + s_L^2 \mathbb{E} \left[ \|W^{(\ell)} z^{(\ell)}\|^2 \right] \quad (2.22)$$

$$= (1 + s_L^2) \|z^{(\ell)}\|^2. \quad (2.23)$$

Therefore

$$\mathbb{E} \|z^{(L)}\|^2 = (1 + s_L^2)^L \|x\|^2. \quad (2.24)$$

Let

$$s_L = L^{-\beta}. \quad (2.25)$$

Then

$$(1 + s_L^2)^L \sim \exp \left( L s_L^2 \right) = \exp \left( L^{1-2\beta} \right). \quad (2.26)$$

This gives three regimes:

|                                                                                                                                                                                                                                 |        |
|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------|
| $\beta < \frac{1}{2} \quad : \quad \text{exploding regime,}$<br>$\beta = \frac{1}{2} \quad : \quad \text{non-trivial diffusive regime,}$<br>$\beta > \frac{1}{2} \quad : \quad \text{identity regime for iid centered layers.}$ | (2.27) |
|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------|

Thus, for standard iid initialization,

|                               |        |
|-------------------------------|--------|
| $s_L \sim \frac{1}{\sqrt{L}}$ | (2.28) |
|-------------------------------|--------|

is the critical scaling.

### 2.4.1 The $1/\sqrt{L}$ limit: Neural SDE

The residual increments have size

$$\Delta z_\ell \sim \frac{1}{\sqrt{L}},$$

while there are  $L$  independent increments. Hence their cumulative fluctuation is order one:

$$\sqrt{L} \times \frac{1}{\sqrt{L}} = O(1). \quad (2.29)$$

This is exactly the central-limit/Brownian scaling.

Schematically,

$$z^{(\ell+1)} = z^{(\ell)} + \frac{1}{\sqrt{L}} G(z^{(\ell)}) \xi_\ell \quad (2.30)$$

converges to a stochastic differential equation

$$\boxed{dz_t = G(z_t) dB_t.} \quad (2.31)$$

Thus the natural continuous-time limit of an iid deep ResNet is generically a *Neural SDE*, rather than a Neural ODE.

### 2.4.2 The $1/L$ limit: Neural ODE

Now suppose instead that the residual vector field changes smoothly with layer number:

$$F^{(\ell)}(z) \approx f\left(\frac{\ell}{L}, z\right). \quad (2.32)$$

Take

$$s_L = \frac{1}{L}. \quad (2.33)$$

Then

$$z^{(\ell+1)} - z^{(\ell)} = \frac{1}{L} f\left(\frac{\ell}{L}, z^{(\ell)}\right). \quad (2.34)$$

Writing

$$t = \frac{\ell}{L}, \quad \Delta t = \frac{1}{L},$$

this is simply the Euler discretization

$$\frac{z^{(\ell+1)} - z^{(\ell)}}{\Delta t} = f(t, z^{(\ell)}). \quad (2.35)$$

Hence

$$\boxed{\frac{dz_t}{dt} = f(t, z_t).} \quad (2.36)$$

The role of increasing  $L$  is now to make the time discretization finer.

**Key point.** There is no contradiction between the Neural SDE and Neural ODE pictures. They correspond to different assumptions about the regularity of the weights across depth:

$$\boxed{\begin{array}{ll} \text{iid / rough across layers} & \implies s_L \sim L^{-1/2} \quad (\text{SDE}), \\ \text{smooth / correlated across layers} & \implies s_L \sim L^{-1} \quad (\text{ODE}). \end{array}} \quad (2.37)$$

In particular, using  $s_L = 1/L$  together with iid, zero-mean residual blocks does *not* produce a non-trivial ODE: the random residuals simply average away and the network approaches the identity map.

### 2.4.3 What lies between $1/\sqrt{L}$ and $1/L$ ?

Suppose

$$s_L = L^{-\beta}, \quad \frac{1}{2} < \beta < 1. \quad (2.38)$$

If the layers remain iid, then

$$Ls_L^2 = L^{1-2\beta} \rightarrow 0, \quad (2.39)$$

so the fluctuations disappear and the network approaches the identity. Thus there is no new non-trivial iid limit between the SDE and ODE scalings.

However, an intermediate regime becomes possible if the weights are correlated across depth. A useful model is fractional Brownian motion. Let  $B_t^H$  have Hurst exponent

$$H \in \left(\frac{1}{2}, 1\right).$$

Its fluctuations over  $L$  increments scale as

$$L^H,$$

rather than  $\sqrt{L}$ . Therefore the corresponding critical residual scale is

$$\boxed{s_L \sim L^{-H}}. \quad (2.40)$$

The two familiar limits are recovered as

$$H = \frac{1}{2} \implies \text{Brownian/SDE scaling}, \quad (2.41)$$

$$H = 1 \implies \text{smooth/ODE scaling}. \quad (2.42)$$

Thus one can think schematically of

$$\boxed{\text{roughness of weights across depth} \leftrightarrow \text{critical depth scaling}}. \quad (2.43)$$

## 2.5 Number of parameters versus number of data points

We now study the simplest supervised-learning model in which a similar dimensionless ratio appears.

Consider linear regression

$$f_\theta(x) = \theta^\top x, \quad \theta \in \mathbb{R}^d, \quad (2.44)$$

with data

$$\mathcal{D} = \{(x_i, y_i)\}_{i=1}^n. \quad (2.45)$$

Assume the isotropic Gaussian model

$$x_i \stackrel{\text{iid}}{\sim} \mathcal{N}(0, I_d), \quad (2.46)$$

$$y_i = x_i^\top \theta_0 + \epsilon_i, \quad (2.47)$$

$$\epsilon_i \stackrel{\text{iid}}{\sim} \mathcal{N}(0, \sigma_\epsilon^2), \quad (2.48)$$

and use an isotropic random-effects model for the true parameter,

$$\theta_0 \sim \mathcal{N}\left(0, \frac{r^2}{d} I_d\right), \quad (2.49)$$

so that typically

$$\|\theta_0\|^2 \approx r^2.$$

The ridge estimator is

$$\hat{\theta}_\lambda = \arg \min_{\theta} \left\{ \frac{1}{n} \|y - X\theta\|_2^2 + \lambda \|\theta\|_2^2 \right\}. \quad (2.50)$$

Its closed-form solution is

$$\boxed{\hat{\theta}_\lambda = \left( \frac{X^\top X}{n} + \lambda I \right)^{-1} \frac{X^\top y}{n}.} \quad (2.51)$$

Therefore the central random object is the sample covariance matrix

$$S = \frac{X^\top X}{n}. \quad (2.52)$$

### 2.5.1 High-dimensional limit and the Marchenko–Pastur law

Take

$$n, d \rightarrow \infty, \quad \boxed{\gamma = \frac{d}{n}} \text{ fixed.} \quad (2.53)$$

The empirical eigenvalue distribution of

$$S = \frac{X^\top X}{n}$$

converges to the Marchenko–Pastur distribution

$$d\mu_\gamma(t) = \rho_\gamma(t) dt + \left(1 - \frac{1}{\gamma}\right)_+ \delta_0, \quad (2.54)$$

where

$$\rho_\gamma(t) = \frac{\sqrt{(\lambda_+ - t)(t - \lambda_-)}}{2\pi\gamma t} \mathbf{1}_{[\lambda_-, \lambda_+]}(t), \quad (2.55)$$

with

$$\boxed{\lambda_\pm = (1 \pm \sqrt{\gamma})^2.} \quad (2.56)$$

Thus the spectrum depends on  $d$  and  $n$  primarily through the single aspect ratio

$$\boxed{\gamma = d/n}.$$

This is analogous to the effective depth  $L/N$  in the previous example.

**Example:**  $d/n = 3$ . For

$$\gamma = 3,$$

we have

$$\lambda_- = (1 - \sqrt{3})^2 \approx 0.536, \quad (2.57)$$

$$\lambda_+ = (1 + \sqrt{3})^2 \approx 7.464. \quad (2.58)$$

Moreover,

$$1 - \frac{1}{\gamma} = \frac{2}{3}. \quad (2.59)$$

Hence  $2/3$  of the eigenvalues are exactly zero in the large-system limit. This simply reflects the fact that when  $d > n$ ,

$$\text{rank}(X^\top X) \leq n,$$

so there are  $d - n$  unconstrained parameter directions.

### 2.5.2 Prediction risk and the optimal ridge parameter

For a fresh input

$$x_{\text{test}} \sim \mathcal{N}(0, I_d),$$

define the excess prediction risk

$$R(\lambda) = \mathbb{E} \left[ \left( x_{\text{test}}^\top (\hat{\theta}_\lambda - \theta_0) \right)^2 \right]. \quad (2.60)$$

In the high-dimensional isotropic limit,

$$R(\lambda) = r^2 \int \frac{\lambda^2}{(t + \lambda)^2} d\mu_\gamma(t) + \sigma_\epsilon^2 \gamma \int \frac{t}{(t + \lambda)^2} d\mu_\gamma(t). \quad (2.61)$$

The two terms have a simple interpretation:

$$\text{bias} = r^2 \int \frac{\lambda^2}{(t + \lambda)^2} d\mu_\gamma(t), \quad (2.62)$$

$$\text{variance} = \sigma_\epsilon^2 \gamma \int \frac{t}{(t + \lambda)^2} d\mu_\gamma(t). \quad (2.63)$$

Differentiating,

$$\frac{dR}{d\lambda} = 2 \left( r^2 \lambda - \sigma_\epsilon^2 \gamma \right) \int \frac{t}{(t + \lambda)^3} d\mu_\gamma(t). \quad (2.64)$$

Therefore

$$\lambda_* = \frac{\sigma_\epsilon^2 \gamma}{r^2}. \quad (2.65)$$

Equivalently, defining

$$\text{SNR} = \frac{r^2}{\sigma_\epsilon^2}, \quad (2.66)$$

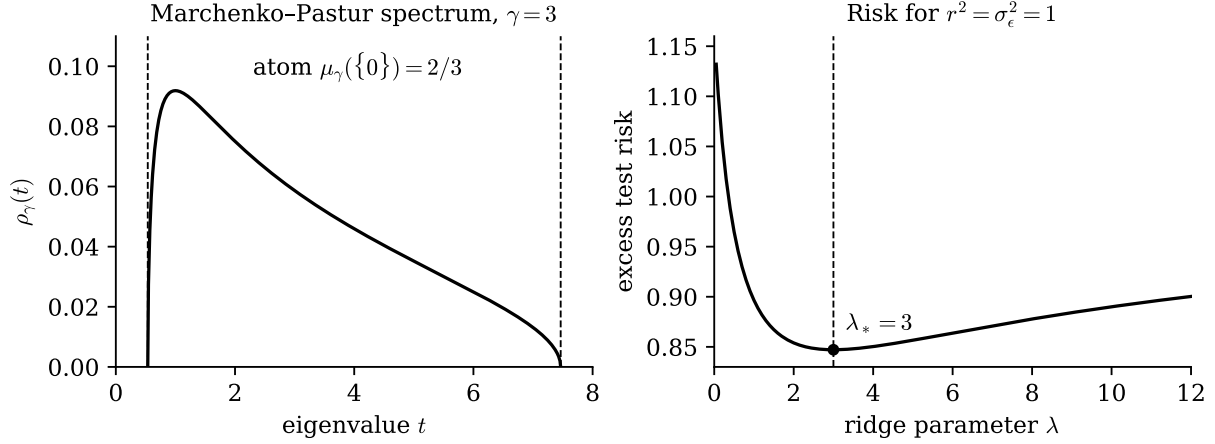


Figure 2.1: High-dimensional linear regression at aspect ratio  $\gamma = d/n = 3$ . Left: the continuous part of the Marchenko–Pastur spectrum; an additional atom of weight  $2/3$  sits at zero. Right: the asymptotic excess prediction risk of ridge regression for  $r^2 = \sigma_\epsilon^2 = 1$ . The minimum occurs at  $\lambda_* = \gamma\sigma_\epsilon^2/r^2 = 3$ .

we obtain the simple scaling law

$$\boxed{\lambda_* = \frac{\gamma}{\text{SNR}}}. \quad (2.67)$$

For example, take

$$\gamma = 3, \quad r^2 = 1, \quad \sigma_\epsilon^2 = 1.$$

Then

$$\boxed{\lambda_* = 3}. \quad (2.68)$$

A numerical sweep of the Marchenko–Pastur prediction-risk formula gives the same minimum.

## 2.6 Summary: the recurring idea

These examples illustrate the same general principle: large systems are often controlled not by the raw sizes individually, but by a small number of *dimensionless ratios*.

|                             |                        |        |
|-----------------------------|------------------------|--------|
| deep linear network         | : $\xi = L/N$ ,        |        |
| ResNet                      | : $LS_L^2$ (iid case), |        |
| high-dimensional regression | : $\gamma = d/n$ .     | (2.69) |

The goal of scaling theory is therefore to identify:

1. the correct macroscopic variables;
2. the dimensionless combinations that remain finite;
3. the corresponding limiting dynamics;
4. the scaling of hyperparameters required to obtain a non-trivial limit.

This is the basic philosophy behind studying neural networks through large- $N$ , large- $L$ , large-data, and large-training-time limits.